

Proyecto: Gestor de Gastos Personales con Análisis

1. Descripción general del proyecto

El proyecto consiste en una aplicación web desarrollada en Python mediante FastAPI que permitirá a un usuario gestionar sus ingresos y gastos personales.

La aplicación tendrá las siguientes funcionalidades principales:

- Registrar ingresos y gastos.
- Editar movimientos existentes.
- Clasificar movimientos por categorías.
- Gestionar plantillas de gastos e ingresos recurrentes con generación automática de movimientos al llegar su fecha.
- Consultar historial de movimientos.
- Filtrar movimientos por fecha, categoría o tipo.
- Mostrar estadísticas y balances.
- Definir presupuestos mensuales por categoría y controlar su cumplimiento.
- Crear objetivos de ahorro con fecha límite y seguimiento de progreso.
- Importar y exportar datos en formato JSON y CSV.
- Guardar toda la información en una base de datos SQLite.

El objetivo principal es ofrecer una herramienta sencilla para controlar las finanzas personales y visualizar cómo se distribuye el dinero.

2. Arquitectura y diseño orientado a objetos

2.1 Jerarquía de clases

La aplicación utilizará programación orientada a objetos y contará con herencia, polimorfismo y clases abstractas.

Clase abstracta: Movimiento

Representa un movimiento financiero genérico.

Atributos

Atributo		Visibilidad	Tipo	Descripción
id	privado	int		Identificador único descripcion
privado	str			Descripción del movimiento cantidad
	privado	float		Cantidad de dinero
fecha	privado	str		Fecha del movimiento
categoria	privado	str		Categoría asociada

Métodos

Constructor

```
Movimiento(descripcion, cantidad, fecha, categoria)
```

Parámetros

- descripcion: texto no vacío.
- cantidad: número positivo.
- fecha: fecha válida.

- categoria: texto no vacío.

Validaciones

- La cantidad no puede ser negativa.
- La descripción no puede estar vacía.

Excepciones

- ValueError si los datos son inválidos.

Método abstracto

`calcular_impacto()`

Tipo de retorno

`float`

Función

Devuelve el impacto del movimiento sobre el balance.

Clase hija: Ingreso

Hereda de Movimiento. Representa un ingreso económico.

Métodos

`calcular_impacto()`

Devuelve la cantidad en positivo.

```
return self.cantidad
```

Clase hija: Gasto

Hereda de Movimiento. Representa un gasto económico.

Métodos

`calcular_impacto()`

Devuelve la cantidad en negativo.

```
return -self.cantidad
```

Clase hija: MovimientoRecurrente

Hereda de Movimiento.

Representa un movimiento que se repite automáticamente con una frecuencia determinada.

Atributos adicionales

Atributo	Visibilidad	Tipo	Descripción
Periodicidad	"semanal" o "mensual"	activo	privado
Indica si la recurrencia está activa	bool		

Métodos

calcular_impacto()

Delega en el tipo base almacenado en el atributo `tipo`.

```
return self.cantidad if self.tipo == "ingreso" else -self.cantidad
```

generar_siguiente_fecha()

Calcula la próxima fecha de generación a partir de la última fecha registrada y la frecuencia configurada.

Retorno

```
str # fecha en formato YYYY-MM-DD
```

Clase: Presupuesto

Representa un límite de gasto mensual definido por el usuario para una categoría concreta.

Atributos

Atributo	Tipo	Descripción
<code>id</code>	int	Identificador único usuario_id
<code>usuario</code>	int	Usuario propietario
<code>limite</code>	float	Categoría a la que aplica el presupuesto
<code>mes</code>	str	Importe máximo permitido en el mes
<code>mes_de_aplicación</code>	str	Mes de aplicación en formato YYYY-MM

Métodos

porcentaje_consumido(gastado)

Calcula el porcentaje del presupuesto ya utilizado.

Parámetros

- gastado: importe total gastado en esa categoría durante el mes.

Retorno

```
float # puede superar 100.0 si se excede el límite
```

esta_superado(gastado)

Devuelve `True` si el gasto supera el límite establecido.

Retorno

```
bool
```

Clase: ObjetivoAhorro

Representa una meta económica que el usuario quiere alcanzar antes de una fecha límite.

Atributos

Atributo	Tipo	Descripción
id	int	Identificador único usuario_id
nombre	int	Usuario propietario
cantidad_meta	str	Nombre descriptivo del objetivo
cantidad_actual	float	Importe total a alcanzar
importe_acumulado_hasta_el_momento	float	Importe acumulado hasta el momento
fecha_limite	str	Fecha máxima en formato YYYY-MM-DD

Métodos

progreso()

Calcula el porcentaje de avance hacia la meta.

Retorno

float # entre 0.0 y 100.0

dias_restantes()

Calcula los días que quedan hasta la fecha límite.

Retorno

int

esta_completado()

Devuelve `True` si la cantidad actual ha alcanzado o superado la meta.

Retorno

bool

Clase: Usuario

Representa al usuario propietario de los movimientos.

Atributos

Atributo	Tipo	Descripción
id	int	Identificador
nombre	str	Nombre del usuario
email	str	Correo electrónico

Métodos

agregar_movimiento(movimiento)

Añade un

movimiento a la colección. **Parámetros**

- movimiento: objeto Movimiento.

Validaciones

- Debe ser una instancia válida de Movimiento.

calcular_balance()

Calcula el balance total utilizando polimorfismo.

Retorno

float

Funcionamiento

Recorre todos los movimientos y llama a:

```
movimiento.calcular_impacto()
```

Gracias al polimorfismo, el comportamiento cambia según si el objeto es Ingreso o Gasto.

obtener_gastos_por_categoria() Agrupa

gastos según su categoría. **Retorno**

dict

Clase: GestorFinanzas

Clase encargada de coordinar las operaciones principales del sistema.

Atributos

Atributo	Tipo
usuarios	list
conexion_db	sqlite3.Connection

La lista de usuarios constituye una estructura dinámica.

Métodos

crear_usuario(nombre, email)

Crea un usuario nuevo.

registrar_ingreso(...)

Crea un objeto Ingreso.

registrar_gasto(...)

Crea un objeto Gasto.

guardar_datos()

Guarda la información en SQLite.

cargar_datos()

Carga los datos desde SQLite.

exportar_json(ruta)

Exporta todos los movimientos a un archivo JSON.

importar_json(ruta)

Importa movimientos desde un archivo JSON.

exportar_csv(ruta)

Exporta todos los movimientos a un archivo CSV con cabecera.

crear_presupuesto(categoria, limite, mes)

Crea un presupuesto mensual para una categoría.

obtener_presupuestos(usuario_id)

Devuelve la lista de presupuestos activos del usuario.

crear_objetivo(nombre, cantidad_meta, fecha_limite)

Crea un nuevo objetivo de ahorro.

obtener_objetivos(usuario_id)

Devuelve la lista de objetivos del usuario con su progreso calculado.

3. Relaciones entre clases

Herencia

```
Movimiento
├── Ingreso
├── Gasto
└── MovimientoRecurrente
```

Composición

- Usuario contiene una colección de movimientos.
 - Usuario contiene una colección de presupuestos.
 - Usuario contiene una colección de objetivos de ahorro.
 - GestorFinanzas contiene usuarios.
-

Polimorfismo

El método:

calcular_impacto()

se comporta de forma diferente dependiendo del tipo de movimiento.

4. Persistencia de datos

La aplicación utilizará SQLite.

Tabla usuarios

```
CREATE TABLE usuarios (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  nombre TEXT NOT NULL,  
  email TEXT NOT NULL  
);
```

Tabla movimientos

```
CREATE TABLE movimientos (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  usuario_id INTEGER NOT NULL,  
  tipo TEXT NOT NULL,  
  descripcion TEXT NOT NULL,  
  cantidad REAL NOT NULL,  
  fecha TEXT NOT NULL,  
  categoria TEXT NOT NULL,  
  recurrente INTEGER NOT NULL DEFAULT 0,  
  frecuencia TEXT,  
  fecha_creacion TEXT,  
  FOREIGN KEY(usuario_id) REFERENCES usuarios(id)  
);
```

Tabla recurrentes

```
CREATE TABLE recurrentes (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  usuario_id INTEGER NOT NULL,  
  tipo TEXT NOT NULL,  
  descripcion TEXT NOT NULL,  
  cantidad REAL NOT NULL,  
  categoria TEXT NOT NULL,  
  frecuencia TEXT NOT NULL,  
  proxima_fecha TEXT NOT NULL,  
  FOREIGN KEY(usuario_id) REFERENCES usuarios(id)  
);
```

Tabla presupuestos

```
CREATE TABLE presupuestos (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  usuario_id INTEGER NOT NULL,  
  categoria TEXT NOT NULL,  
  limite REAL NOT NULL,  
  mes TEXT NOT NULL,  
  FOREIGN KEY(usuario_id) REFERENCES usuarios(id)  
);
```

Tabla objetivos_ahorro

```
CREATE TABLE objetivos_ahorro (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```

    usuario_id INTEGER NOT NULL,
    nombre TEXT NOT NULL,
    cantidad_meta REAL NOT NULL,
    cantidad_actual REAL NOT NULL DEFAULT 0,
    fecha_limite TEXT NOT NULL,
    FOREIGN KEY(usuario_id) REFERENCES usuarios(id)
);

```

Tabla categorías

```

CREATE TABLE categorias (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre TEXT NOT NULL,
    usuario_id INTEGER NOT NULL DEFAULT 0,
    es_default INTEGER NOT NULL DEFAULT 0,
    tipo TEXT NOT NULL DEFAULT 'gasto',
    UNIQUE(nombre, usuario_id, tipo)
);

```

Las categorías con `usuario_id = 0` y `es_default = 1` son las predefinidas del sistema, insertadas con `INSERT OR IGNORE` al inicializar la BD. Las categorías personalizadas usan el `usuario_id` del usuario y `es_default = 0`.

Existen dos conjuntos de categorías predefinidas separados por tipo:

- **Gastos** (10): Comida, Transporte, Vivienda, Salud, Ocio, Ropa, Educación, Tecnología, Suscripciones, Otros.
- **Ingresos** (8): Salario, Freelance, Inversiones, Alquiler, Venta, Regalo, Reembolso, Otros.

La restricción `UNIQUE(nombre, usuario_id, tipo)` permite que exista “Otros” tanto en gastos como en ingresos sin conflicto.

5. Formatos de exportación

JSON

```

{
  "usuario": "Carlos",
  "movimientos": [
    {
      "tipo": "gasto",
      "descripcion": "Supermercado",
      "cantidad": 50,
      "fecha": "2026-05-14",
      "categoria": "Comida",
      "recurrente": false,
      "frecuencia": null
    }
  ]
}

```

CSV

El archivo exportado incluirá una fila de cabecera con los nombres de los campos:

```

tipo,descripcion,cantidad,fecha,categoria,recurrente,frecuencia
gasto,Supermercado,50.0,2026-05-14,Comida,false,
ingreso,Nómina,1800.0,2026-05-01,Salario,true,mensual

```

6. Programa principal

La aplicación seguirá una arquitectura desacoplada cliente-servidor.

El backend será una API REST desarrollada en Python utilizando FastAPI y el frontend será una aplicación independiente desarrollada con Vue.js.

La comunicación entre frontend y backend se realizará mediante peticiones HTTP utilizando JSON.

6.1 Arquitectura del sistema

```
Frontend (Vue.js)
  ↓ HTTP/JSON
Backend API REST (FastAPI)
  ↓
SQLite
```

6.2 Estructura del proyecto

```
proyecto/
├── backend/
│   ├── app/
│   │   ├── models/
│   │   ├── routes/
│   │   ├── services/
│   │   ├── database/
│   │   ├── schemas/
│   │   └── main.py
│   ├── requirements.txt
│   └── database.db
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   ├── views/      ← Dashboard, Movimientos, Recurrentes, Estadísticas, Objetivos
│   │   ├── services/
│   │   └── App.vue
│   ├── package.json
└── README.md
```

6.3 Backend API REST

El backend se implementará utilizando FastAPI. La API

será responsable de:

- Gestionar usuarios.
- Gestionar movimientos.
- Validar datos.
- Acceder a SQLite.
- Importar/exportar JSON.
- Calcular estadísticas.

Endpoints principales

Movimientos

Método	Endpoint	Descripción
GET	/movimientos	Obtener movimientos. Query params opcionales: usuario_id, tipo, categoria, fecha_desde, fecha_hasta, recurrente (bool)

POST	/movimientos	Crear movimiento
PUT	/movimientos/{id}	Editar movimiento
DELETE	/movimientos/{id}	Eliminar movimiento

Los endpoints `/ingresos` y `/gastos` fueron descartados durante el desarrollo en favor de un único endpoint `/movimientos` con filtrado por tipo.

Recurrentes

Método	Endpoint	Descripción
GET	/recurrentes	Obtener plantillas del usuario. Genera automáticamente los movimientos vencidos antes de devolver la lista. Query param: <code>usuario_id</code>
POST	/recurrentes	Crear plantilla
PUT	/recurrentes/{id}	Editar plantilla
DELETE	/recurrentes/{id}	Eliminar plantilla (los movimientos generados no se borran)

Estadísticas

Método	Endpoint
GET	/estadisticas/balance
GET	/estadisticas/categorias

Presupuestos

Método	Endpoint	Descripción
GET	/presupuestos	Obtener presupuestos del usuario
POST	/presupuestos	Crear presupuesto mensual por categoría
DELETE	/presupuestos/{id}	Eliminar presupuesto

Objetivos de ahorro

Método	Endpoint	Descripción
GET	/objetivos	Obtener objetivos del usuario
POST	/objetivos	Crear nuevo objetivo de ahorro
PUT	/objetivos/{id}/aportacion	Registrar una aportación al objetivo
DELETE	/objetivos/{id}	Eliminar objetivo

JSON y CSV

Método	Endpoint	Descripción
POST	/importar-json	Importar movimientos desde JSON
GET	/exportar-json	Exportar movimientos a JSON
GET	/exportar-csv	Exportar movimientos a CSV

Categorías

Método	Endpoint	Descripción
GET	/categorias	Obtener categorías del usuario + predefinidas. Acepta query param opcional <code>tipo</code> (<code>gasto</code> o <code>ingreso</code>) para filtrar
POST	/categorias	Crear categoría personalizada. Body requiere <code>nombre</code> , <code>usuario_id</code> y <code>tipo</code>
DELETE	/categorias/{id}	Eliminar categoría (solo personalizadas). Reasigna sus movimientos a "Otros"

Las categorías predefinidas no pueden ser eliminadas y se identifican con `usuario_id = 0` y `es_default = 1`. Existen categorías predefinidas para `tipo = 'gasto'` y para `tipo = 'ingreso'` por separado.

6.4 Frontend

La interfaz gráfica será una aplicación web desarrollada con Vue.js.

El frontend consumirá la API REST mediante peticiones HTTP utilizando Axios.

Pantallas principales

Dashboard principal

Mostrará:

- Balance total
 - Últimos movimientos
 - Resumen mensual
 - Gráficas de evolución
 - Resumen de objetivos de ahorro activos con barra de progreso
-

Página de movimientos

Permitirá:

- Registrar ingresos y gastos
 - Seleccionar categoría desde un desplegable (evita erratas)
 - Marcar movimientos como recurrentes al crearlos (con frecuencia semanal o mensual)
 - Editar movimientos existentes mediante modal
 - Eliminar movimientos con confirmación
 - Filtrar por fecha, tipo y categoría
 - Importar y exportar datos en JSON y CSV
 - **Gestionar categorías:** panel con tabs **Gastos / Ingresos** para ver y crear categorías de cada tipo por separado. Las categorías personalizadas son eliminables (sus movimientos se reasignan a “Otros”). El selector de categoría en los formularios carga solo las del tipo del movimiento (gasto o ingreso) y se actualiza automáticamente al cambiar el tipo.
-

Página de recurrentes

Gestor de **plantillas** de gastos e ingresos que se repiten periódicamente. Al cargar la página, el backend comprueba qué plantillas tienen `proxima_fecha <= hoy`, genera los movimientos pendientes en la tabla `movimientos` (con `recurrente=1`) y avanza la fecha al siguiente periodo.

Permitirá:

- Ver las plantillas separadas por tabs **Ingresos / Gastos** con contador por tipo
 - Consultar descripción, categoría, cantidad, frecuencia (semanal/mensual) y próxima fecha de cada plantilla
 - **Crear** nuevas plantillas con modal (tipo, descripción, categoría, cantidad, frecuencia, próxima fecha)
 - **Editar** cualquier campo de la plantilla
 - **Eliminar** una plantilla con confirmación — los movimientos ya generados no se borran
 - Empty state cuando no hay plantillas del tipo seleccionado
-

Página de estadísticas

Mostrará:

- Gastos por categoría con barras de progreso

- Balance mensual con gráfica de barras
- Porcentajes de distribución
- Presupuestos mensuales por categoría con indicador de consumo
- Gráficas dinámicas implementadas con Chart.js

Página de objetivos de ahorro

Permitirá:

- Crear nuevos objetivos con nombre, importe meta y fecha límite
- Registrar aportaciones parciales a cada objetivo
- Visualizar el progreso de cada objetivo con barra y porcentaje
- Eliminar objetivos completados o cancelados

7. Tecnologías utilizadas

Tecnología	Uso
Python	Lógica del programa
FastAPI	API REST backend Vue.js
	Interfaz web frontend
SQLite	Base de datos
Axios	Comunicación frontend-backend
Chart.js	Gráficas y estadísticas HTML/CSS
Diseño de interfaz	
JSON	Importación/exportación
CSV	Exportación tabular de movimientos

8. Requisitos mínimos cubiertos

Requisito	Implementación
Herencia	Movimiento → Ingreso / Gasto / MovimientoRecurrente
Clase abstracta	Movimiento
Polimorfismo	calcular_impacto()
Estructuras dinámicas	Listas de movimientos, presupuestos y objetivos
Persistencia SQLite	Tablas movimientos, presupuestos, objetivos_ahorro, categorias
Importar/exportar JSON	Endpoints /importar-json y /exportar-json
Exportar CSV	Endpoint /exportar-csv CRUD
completo de movimientos	GET / POST / PUT / DELETE
Movimientos recurrentes	Tabla <code>recurrentes</code> (plantillas) + auto-generación en GET /recurrentes + página /recurrentes con CRUD completo y tabs Ingresos/Gastos
Presupuestos por categoría de ahorro	Clase Presupuesto + endpoints /presupuestos Objetivos Clase ObjetivoAhorro + endpoints /objetivos
Gestión de categorías	Tabla categorias + 18 predefinidas (10 gasto + 8 ingreso) separadas por tipo + personalizadas + CategoriaSelect.vue filtra por tipo
Interfaz web	Vue.js + FastAPI

9. Posibles mejoras futuras

- Sistema de autenticación con múltiples usuarios y contraseña.
- Notificaciones visuales al superar un presupuesto mensual.
- Exportación a PDF con informe de gastos mensual.

- Predicción de gastos futuros basada en el historial.
- Soporte para múltiples divisas con conversión automática.
- Modo PWA (Progressive Web App) para uso desde móvil sin conexión.